

CHAPTER 07

07. ORM(Sequelize) + 게시판 — SQL을 메서드로 바꾸다

Node.js · Express · NestJS 강의 · 2026

6장에서는 `db.prepare("SELECT * FROM posts WHERE id = ?")` 처럼 SQL을 문자열로 손수 적었다. 테이블도 `CREATE TABLE posts (. . .)` 를 직접 썼고, 조회 결과는 그냥 평범한 객체였다. 이 장은 그 SQL 문자열을 전부 걷어낸다. 대신 자바스크립트로 **모델**을 선언하면, 그 모델이 `create · findAll · save · destroy` 같은 메서드를 제공한다. 이렇게 객체 세계와 테이블 세계를 잇는 도구가 **ORM(Object-Relational Mapping)**이고, Node.js에서 가장 오래된 ORM이 **Sequelize**다.

DB는 6장과 똑같은 SQLite, 만드는 것도 똑같은 게시판이다. 달라지는 건 단 하나, **"SQL 문자열을 우리가 쓰느냐, Sequelize가 대신 만드느냐"** 다. 같은 게시판을 두 방식으로 만들어 보면 ORM이 무엇을 덜어 내는지 또렷하게 보인다.

이 자료는 **개념 연습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- **모델 정의** — `sequelize.define` 으로 테이블 모양을 자바스크립트로 선언하기 (`CREATE TABLE` 을 대신한다)
- **CRUD를 메서드로** — `create · findAll · findByPk · save · destroy` 로 SQL 없이 데이터 다루기
- **비동기(async/await)** — ORM 호출이 왜 `await` 를 달고 다니는지
- **:memory: vs 파일 DB** — 휘발성 메모리 DB와, 서버를 꺼도 남는 파일 DB
- **관계** — `hasMany / belongsTo` 로 테이블을 잇고, `include` 한 줄로 작성자·댓글을 한 번에 가져오기
- **createdAt / updatedAt** — ORM이 알아서 붙이고 관리하는 시각 컬럼

내용은 세 주제로 나뉜다. 먼저 모델과 CRUD로 감을 잡고, 같은 게시판을 ORM으로 다시 만든 뒤, ORM의 진짜 강점인 관계를 본다. 앞의 둘은 서버 없이 콘솔에 결과를 찍어 보며 익히고, 게시판 API에서만 포트 `3000` 에서 도는 서버가 등장한다.

주제	핵심
Sequelize 기초	모델 정의 + CRUD를 메서드로 (6장 날퀴리와 1:1 비교)
게시판 API	같은 게시판 REST API를 ORM으로 (서버 + 파일 DB)
관계	<code>hasMany / belongsTo</code> + <code>include</code> 로 작성자·댓글 한 번에

핵심 개념 — ORM은 "객체와 테이블 사이의 통역사"다

데이터베이스의 세계는 **테이블**이다. 행과 열, 그리고 그것을 다루는 언어는 SQL이다. 한편 우리가 코드를 쓰는 세계는 **객체**다. `post.title`, `post.author` 같은 속성으로 데이터를 만진다. 이 둘은 모양이 다르다. 그래서 6장에서는 매번 우리가 직접 번역했다 — 객체에서 값을 꺼내 `?` 자리에 끼워 넣고(`INSERT`), 돌아온 행을 다시 객체로 받아 썼다.

ORM(Object-Relational Mapping, 객체-관계 매핑)은 그 번역을 대신하는 통역사다. 우리는 객체로 말하고, ORM이 그 말을 SQL로 옮겨 DB에 전한다. DB가 행으로 답하면, ORM이 그걸 다시 객체로 옮겨 우리에게 건넨다. 우리가 SQL을 한 줄도 쓰지 않아도 데이터가 오간다.

구체적으로 무엇이 바뀌는지 6장과 나란히 두면 이렇다.

하는 일	6장 (raw SQL)	7장 (Sequelize ORM)
테이블 만들기	<code>CREATE TABLE posts (. . .) 문자열</code>	<code>sequelize.define("Post", {...})</code>
한 건 넣기	<code>INSERT INTO.. VALUES (?, ?, ?)</code>	<code>Post.create({...})</code>
전체 조회	<code>SELECT * FROM posts</code>	<code>Post.findAll()</code>
id로 한 건 조회	<code>SELECT * FROM posts WHERE id = ?</code>	<code>Post.findById(id)</code>
수정	<code>UPDATE posts SET... WHERE id = ?</code>	인스턴스 값 변경 후 <code>post.save()</code>
삭제	<code>DELETE FROM posts WHERE id = ?</code>	<code>post.destroy()</code>
작성 시각	<code>created_at</code> 컬럼을 직접 선언	<code>createdAt / updatedAt</code> 자동

그래서 **SQL은 안 배워도 되나** — 아니다. ORM은 SQL을 숨기는 도구이지 없애는 도구가 아니다. Sequelize도 결국 안에서 SQL을 만들어 SQLite에 보낸다. 어떤 메서드가 어떤 쿼리로 번역되는지 감이 있어야, 느린 쿼리를 만났을 때 원인을 짚는다. 6장을 먼저 둔 이유가 이것이다. `logging: false` 를 `console.log` 로 바꾸면 Sequelize가 만든 SQL이 그대로 터미널에 찍힌다 — 한 번쯤 켜고 보면 통역 과정을 확인할 수 있다.

왜 하필 Sequelize인가 — ORM은 하나가 아니다. Sequelize는 가장 오래되고 성숙한 Node.js ORM이라 운영 중인 실무 코드에서 자주 만난다. 다만 신규 프로젝트는 타입스크립트 지원이 더 좋은 Prisma·Drizzle 쪽으로 옮겨 가는 추세다. 이 책에서 Sequelize는 "ORM이란 무엇인가"를 직관적으로 익히는 도구이고, 뒤의 NestJS 파트에서 Prisma로 같은 개념을 다시 다룬다. 여기서 익힌 모델·관계·추상화 감각은 그대로 Prisma로 이어진다. 도구만 바뀔 뿐이다.

준비

예제 폴더에서 `npm install` 로 의존성을 설치한다. 의존성은 셋이다. `express` (서버), `sequelize` (SQL을 만드는 ORM 본체), `sqlite3` (Sequelize가 SQLite와 대화할 때 쓰는 드라이버). Sequelize는 여러 DB를 지원하는 ORM이고, 그중 어떤 DB를 쓸지는 드라이버로 정한다 — 여기서는 `sqlite3` 다. MySQL이라면 `mysql2`, PostgreSQL이라면 `pg` 로 바꾼다. **DB를 바꿔도 우리가 쓴 모델 코드는 거의 그대로다.** 이게 ORM의 또 다른 이점이다.

기초 예제와 관계 예제는 콘솔에 결과만 찍고 끝나는 스크립트다. 게시판 API만 서버라서, 띄운 뒤 다른 터미널에서 `curl` 로 요청을 보내면 응답이 온다.

Sequelize 기초 — 모델과 CRUD의 기본기

서버를 만들기 전에, ORM이 SQL을 어떻게 대신하는지부터 콘솔에서 확인한다. 6장의 raw SQL 기초와 같은 흐름(만들고·읽고·고치고·지우기)을 이번엔 SQL 한 줄 없이 한다.

연결 — SQLite에 붙되 SQL은 쓰지 않는다

```
const { Sequelize, DataTypes } = require("sequelize");

// SQLite에 연결 (06과 같은 SQLite지만, 이제 SQL을 직접 쓰지 않는다)
const sequelize = new Sequelize({
  dialect: "sqlite",
  storage: ":memory:", // 메모리 DB(연습용). 프로그램이 끝나면 사라진다.
  logging: false, // Sequelize가 만드는 SQL 로그 끄기 (보고 싶으면 console.log로 바꿔보기)
});
```

`new Sequelize({.})` 가 DB 연결의 출발점이다. 6장의 `new Database(":memory:")` 자리에 해당한다.

- `dialect: "sqlite"` — 어떤 종류의 DB인지 알린다. Sequelize는 같은 코드로 여러 DB를 다루므로, 방언(dialect)을 지정해 줘야 한다. 이 값을 `"mysql"` 로 바꾸면 나머지 모델 코드는 거의 그대로 두고 MySQL로 갈아탈 수 있다.
- `storage: ":memory:"` — 데이터를 어디에 둘지 정한다. `:memory:` 는 메모리에만 존재하는 임시 DB라 프로그램이 끝나면 사라진다. 연습용으로 깔끔하다.
- `logging: false` — Sequelize는 기본적으로 자신이 만든 SQL을 콘솔에 찍는다. 지금은 결과만 보려고 껐다. `console.log` 로 바꾸면 통역된 SQL이 그대로 보인다.

모델 정의 — CREATE TABLE 을 자바스크립트로

```
// 모델 정의 = 테이블 모양을 JS로 선언한다. (CREATE TABLE을 대신해준다)
const Post = sequelize.define("Post", {
  title: { type: DataTypes.STRING, allowNull: false },
  content: { type: DataTypes.TEXT, allowNull: false },
  author: { type: DataTypes.STRING },
});
// → posts 테이블이 만들어지고, id / createdAt / updatedAt 은 Sequelize가 자동 추가한다.
```

여기가 ORM의 심장이다. 6장에서 손으로 쓰던 이 SQL과 비교해 보자.

```
CREATE TABLE posts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  content TEXT NOT NULL,
  author TEXT,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
)
```

`sequelize.define` 은 이 `CREATE TABLE` 을 자바스크립트 객체로 옮긴 것이다. 한 줄씩 짚어 보자.

- `"Post"` — **모델 이름**이다. 단수·대문자로 쓰는 게 관례다. Sequelize는 이 이름을 복수형으로 바꿔 **posts** 테이블과 짝지운다. 모델은 단수(`Post`), 테이블은 복수(`posts`) — 이 규칙을 기억해 두면 헷갈리지 않는다.
- `type: DataTypes.STRING` — `TEXT NOT NULL` 의 `TEXT` 에 해당한다. `STRING` 은 짧은 문자열(SQL의 `VARCHAR`), `TEXT` 는 긴 본문용 문자열이다. 그 밖에 `INTEGER` , `BOOLEAN` , `DATE` 등이 있다.
- `allowNull: false` — `NOT NULL` 을 그대로 옮긴 것이다. 빈 값을 허용하지 않는다. `author` 에는 이 옵션이 없으니 비워 둘 수 있다(`TEXT` 만 있고 `NOT NULL` 이 없는 것과 같다).
- **자동으로 붙는 세 컬럼** — `id` , `createdAt` , `updatedAt` 은 우리가 적지 않아도 Sequelize가 알아서 넣는다. 6장에서는 `id INTEGER PRIMARY KEY AUTOINCREMENT` 와 `created_at` 을 직접 써야 했다. 게다가 `updatedAt` 은 6장엔 아예 없던 컬럼이다 — 행이 수정될 때마다 ORM이 값을 갱신한다.

짚고 가기 — `sequelize.define` 은 테이블을 선언할 뿐, 아직 DB에 만들지는 않는다. 실제 생성은 잠시 뒤 `sequelize.sync()` 가 한다. "모양을 정의하는 일"과 "DB에 반영하는 일"이 나뉘어 있다.

비동기 — `await` 가 등장하는 이유

```
// await는 async 함수 안에서만 쓸 수 있어서, 전체를 async 함수(main)로 감싸 맨 끝에서 호출한다.
async function main() {
  await sequelize.sync(); // 모델대로 테이블 생성
  //...
}
main();
```

6장의 `better-sqlite3` 는 동기였다. `db.prepare(.).get(1)` 이라고 쓰면 결과가 그 자리에서 바로 나왔다. 하지만 Sequelize의 호출은 비동기다. 결과를 즉시 주지 않고, "나중에 완료되는 약속"(Promise)을 먼저 돌려준다. 그래서 결과가 필요한 곳마다 `await` 를 붙여 끝날 때까지 기다린다.

`await` 는 `async` 로 선언한 함수 안에서만 쓸 수 있다. 그래서 작업 전체를 `async function main()` 으로 감싸고 맨 끝에서 `main()` 을 호출한다. 이 패턴은 이 장 내내, 그리고 앞으로의 모든 DB 코드에서 반복된다.

왜 ORM은 비동기인가 — 진짜 운영 환경의 DB는 보통 다른 서버에 있다. 쿼리를 보내고 응답이 네트워크를 건너 오기까지 시간이 걸린다. 그 사이 서버가 멈춰 서면 다른 요청을 못 받는다. 그래서 "기다리는 동안 다른 일을 하

라"는 의미로 비동기로 설계됐다. SQLite는 파일이라 사실 빠르지만, Sequelize는 모든 DB를 같은 방식으로 다루므로 비동기로 통일돼 있다.

CRUD — 메서드 네 개로

```
// CREATE - INSERT 문 대신 .create()
await Post.create({ title: "첫 번째 글", content: "안녕하세요", author: "김철수" });
await Post.create({ title: "두 번째 글", content: "반갑습니다", author: "이영희" });

// READ - SELECT 대신 .findAll() / .findPk()
const all = await Post.findAll();
console.log("전체 글 수:", all.length);
// .toJSON() = 인스턴스에서 순수 데이터만 꺼낸다(그냥 찍으면 Sequelize 내부 정보가 섞여 지저분하다)
const first = await Post.findPk(1);
console.log("1번 글:", first.toJSON());

// UPDATE - 인스턴스 값을 바꾸고 .save()
const post = await Post.findPk(1);
post.title = "제목 수정됨";
await post.save();
console.log("수정 후 1번 글 제목:", (await Post.findPk(1)).title);

// DELETE - .destroy()
await Post.destroy({ where: { id: 2 } });
console.log("삭제 후 전체 글 수:", await Post.count());
```

6장에서 SQL 문자열을 적던 자리가 전부 메서드로 바뀌었다.

- **Post.create({})** — INSERT 를 대신한다. 객체를 넘기면 그걸 한 행으로 만들어 넣는다. 6장처럼 ? 자리표시자에 값을 순서대로 끼워 넣을 필요가 없다. 키 이름이 컬럼 이름이라 어느 값이 어디로 들어가는지 한눈에 보인다.
- **Post.findAll()** — SELECT * FROM posts 다. 모든 행을 인스턴스 배열로 돌려준다. .length 로 건수를 센다.
- **Post.findPk(1)** — SELECT * FROM posts WHERE id = 1 이다. Pk 는 기본 키(Primary Key, 곧 id)라는 뜻이라, id 한 건을 찾을 때 가장 간결하다.
- **.toJSON()** — findPk 가 돌려주는 건 평범한 객체가 아니라 Sequelize 인스턴스다. 안에 내부 상태가 잔뜩 들어 있어 그냥 console.log 하면 지저분하다. .toJSON() 은 거기서 순수한 데이터만 뽑아낸다. (참고로 res.json(post) 로 응답할 때는 Sequelize가 알아서 이 변환을 하므로 직접 부를 일이 드물다.)
- 수정은 두 단계다 — 먼저 findPk 로 행을 인스턴스로 가져와 post.title = ... 처럼 자바스크립트 객체를 다루듯 값을 바꾼다. 그다음 **post.save()** 를 불러야 비로소 그 변화가 UPDATE 문이 되어 DB에 반영된다. save() 를 빠뜨리면 메모리 속 객체만 바뀌고 DB는 그대로다 — 흔한 실수다.
- **Post.destroy({ where: { id: 2 } })** — DELETE FROM posts WHERE id = 2 다. where 로 조건을 객체로 적는다. 6장의 WHERE id = ? 에 해당하는 부분이 이 where 객체다.
- **Post.count()** — SELECT COUNT(*) 다. 6장에서 .get().n 으로 꺼내던 건수를 곧장 숫자로 돌려준다.

같은 일을 하지만, 문자열로 SQL을 조립하던 손길이 전부 사라졌다. 오히려 `SELECT` 라 쓰거나 따옴표 짝을 빠뜨릴 일도 없다. 이게 ORM이 덜어 내는 가장 직접적인 짐이다.

게시판 API — 같은 게시판, 이번엔 ORM으로

이제 진짜 서버다. 6장의 게시판과 엔드포인트도 상태코드도 똑같은 REST API를, 속만 ORM으로 바꿔 만든다. 두 게시판을 나란히 띄우고 같은 `curl` 을 던지면 거의 같은 응답이 온다 — 다른 점은 ORM이 자동으로 붙인

`createdAt` / `updatedAt` 이 더 보인다는 것뿐이다.

설정 — 파일 DB로 데이터를 남긴다

```
const express = require("express");
const { Sequelize, DataTypes } = require("sequelize");
const path = require("path");

const app = express();
app.use(express.json());

// 파일 기반 DB — board.db 에 저장된다. (서버를 꺼도 데이터가 남음)
const sequelize = new Sequelize({
  dialect: "sqlite",
  storage: path.join(__dirname, "board.db"),
  logging: false,
});

const Post = sequelize.define("Post", {
  title: { type: DataTypes.STRING, allowNull: false },
  content: { type: DataTypes.TEXT, allowNull: false },
  author: { type: DataTypes.STRING },
});
```

기초 예제와 거의 같지만 `storage` 가 다르다. `":memory:"` 가 아니라 **board.db** 파일이다. `path.join(__dirname, "board.db")` 로 현재 파일과 같은 폴더에 `board.db` 를 만든다(`__dirname` 은 현재 파일이 있는 폴더). 메모리 DB는 프로그램이 끝나면 사라지지만, 파일 DB는 디스크에 남으므로 서버를 껐다 켜도 글이 그대로 있다. 진짜 게시판이 되려면 이쪽이어야 한다.

:memory: 와 파일, 언제 무엇을 — 메모리 DB는 실행할 때마다 깨끗한 상태로 시작하니 연습·테스트에 좋다(기초·관계 예제가 이걸 쓴다). 파일 DB는 데이터가 누적되니 실제 저장에 필요할 때 쓴다(게시판 API). 단 `board.db` 는 코드가 아니라 데이터이므로 `.gitignore` 에 `*.db` 로 넣어 저장소에 올라가지 않게 막아 둔다.

앞쪽 `app`, `express.json()` 미들웨어는 4장에서 본 그대로다. `express.json()` 이 있어야 `POST` / `PUT` 의 `req.body` 가 채워진다.

목록·단건 조회 — `findAll` 과 `findByPrimaryKey`

```
// 목록 조회
app.get("/posts", async (req, res) => {
  const posts = await Post.findAll({ order: [["id", "DESC"]] });
  res.json(posts);
});

// 단건 조회
app.get("/posts/:id", async (req, res) => {
  const post = await Post.findById(req.params.id);
  if (!post) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  res.json(post);
});
```

핸들러가 `async` 로 바뀐 것에 주목하자. 안에서 `await` 를 쓰려면 핸들러 자체가 `async` 여야 한다. 4장·6장의 동기 핸들러와 달라진 거의 유일한 형태다.

- `findAll({ order: [["id", "DESC"]] })` — `SELECT * FROM posts ORDER BY id DESC` 다. 6장에서 SQL 끝에 `ORDER BY id DESC` 를 붙이던 자리가 `order` 옵션이 됐다. 형식이 조금 낯설다 — `[["컬럼", "방향"]]` 처럼 배열의 배열인데, 여러 컬럼으로 정렬할 수 있어서 이런 모양이다.
- `findById(req.params.id)` — `id` 한 건 조회다. 여기서 6장과의 미묘한 공통점이 하나 있다. `req.params.id` 는 문자열 `"3"` 이지만, 변환 없이 그대로 넘긴다. 4장의 인메모리 배열에서는 `Number(. )` 로 바꿔야 했지만, DB 쪽(6장의 SQLite도, 여기 Sequelize도)은 `id` 가 정수 컬럼이라 문자열을 알아서 숫자로 맞춰 비교한다.
- **404 처리** — 못 찾으면 `findById` 가 `null` 을 돌려준다. 그때 상태코드 404를 명시한다. 이 분기 로직은 6장과 글자 그대로 같다.

응답에서 눈여겨볼 점은 `res.json(post)` 다. `post` 는 Sequelize 인스턴스인데, `res.json` 이 직렬화하면서 알아서 순수 데이터로 바꿔 준다. 그 결과에 `createdAt / updatedAt` 이 자동으로 끼어 나온다.

생성 — `create`

```
// 생성
app.post("/posts", async (req, res) => {
  const { title, content, author } = req.body;
  if (!title || !content) {
    return res.status(400).json({ message: "title과 content는 필수입니다." });
  }
  const post = await Post.create({ title, content, author: author || "익명" });
  res.status(201).json(post);
});
```

검증(`title · content` 필수 → 400)과 기본값(`author || "익명"`)은 4장·6장과 똑같다. 달라진 건 마지막 두 줄뿐이다.

6장에서는 생성이 두 단계였다. `INSERT` 로 넣고, 방금 만들어진 행의 `id` 를 알아내(`info.lastInsertRowid`) 다시 `SELECT` 로 조회해 돌려줬다. 자동으로 부여된 `id` 를 클라이언트에 알려 주려면 그 왕복이 필요했다.

```
// 6장 - 넣고 다시 조회
const info = db.prepare("INSERT INTO posts.(.) VALUES (?, ?, ?)").run(title, content,
author || "익명");
const created = db.prepare("SELECT * FROM posts WHERE id = ?").get(info.lastInsertRowid);
res.status(201).json(created);
```

ORM은 이 왕복을 한 줄로 줄인다. `Post.create({})` 가 돌려주는 객체에는 부여된 `id` 와 `createdAt` / `updatedAt` 이 이미 채워져 있다. 다시 조회할 필요가 없다. 그대로 201과 함께 돌려준다.

수정 — `save`

```
// 수정 - 보낸 필드만 바꾼다.
app.put("/posts/:id", async (req, res) => {
  const post = await Post.findByPk(req.params.id);
  if (!post) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  if (req.body.title !== undefined) post.title = req.body.title;
  if (req.body.content !== undefined) post.content = req.body.content;
  if (req.body.author !== undefined) post.author = req.body.author;
  await post.save();
  res.json(post);
});
```

`≠ undefined` 로 검사해 **보낸 필드만 바꾸는** 부분 수정의 요령은 4장에서 익힌 그대로다. "값을 보냈는가"와 "값이 참인가"는 다르므로, 빈 문자열로 제목을 지우려는 의도까지 살리려면 `≠ undefined` 여야 한다.

ORM다운 점은 수정 방식이다. 6장에서는 보낸 값과 기존 값을 일일이 골라 합친 뒤 `UPDATE.. SET title=?, content=?, author=?` 로 모든 컬럼을 다시 써야 했다.

```
// 6장 - 기존 값과 합쳐 통째로 UPDATE
const title = req.body.title !== undefined ? req.body.title : post.title;
const content = req.body.content !== undefined ? req.body.content : post.content;
const author = req.body.author !== undefined ? req.body.author : post.author;
db.prepare("UPDATE posts SET title=?, content=?, author=? WHERE id=?").run(title, content,
author, req.params.id);
```

ORM에서는 가져온 인스턴스의 속성을 자바스크립트 객체 다루듯 바꾸고 `post.save()` 만 부르면 된다. Sequelize는 실제로 바뀐 컬럼이 무엇인지 기억했다가 그 부분만 골라 `UPDATE` 한다. 우리가 기존 값을 끌어와 합칠 필요가 없다. `save()` 를 부르는 순간 `updatedAt` 도 자동으로 갱신된다.

삭제 — `destroy`

```
// 삭제
app.delete("/posts/:id", async (req, res) => {
  const post = await Post.findByPk(req.params.id);
  if (!post) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  await post.destroy();
  res.json({ message: "삭제됨", post });
});
```

먼저 `findByPk` 로 행을 찾는다. 없으면 404. 찾았으면 그 인스턴스의 `post.destroy()` 가 `DELETE FROM posts WHERE id = ?` 를 만들어 지운다. 응답에는 지운 내용을 담아 보내는데, 이미 인스턴스로 들고 있으니 6장처럼 삭제 전에 따로 조회해 둘 필요가 없다.

인스턴스 메서드 vs 모델 메서드 — `Post.destroy({ where: { .{. } })` (앞서 본 모델 메서드)와 `post.destroy()` (여기 인스턴스 메서드) 둘 다 삭제다. 차이는 "무엇에 대고 부르느냐"다. 모델(`Post`)에 대고 부르면 조건으로 한꺼번에 지우고, 특정 인스턴스(`post`)에 대고 부르면 그 한 행을 지운다. `save · destroy` 는 인스턴스에, `create · findAll · findByPk · count` 는 모델에 대고 부른다.

서버 시작 — `sync` 먼저, 그다음 `listen`

```
// 테이블을 준비한 뒤(sync) 서버를 시작한다. (기초 예제와 같은 async 함수 패턴)
async function main() {
  await sequelize.sync();
  // 처음 실행 시 샘플 데이터를 한 번만 넣는다(이미 있으면 건너뛴).
  if ((await Post.count()) === 0) {
    await Post.bulkCreate([
      { title: "첫 번째 글", content: "Sequelize로 저장된 글입니다.", author: "김철수" },
      { title: "두 번째 글", content: "서버를 꺼도 남아 있어요.", author: "이영희" },
    ]);
  }
  app.listen(3000, () => console.log("http://localhost:3000 에서 실행 중"));
}
main();
```

순서가 핵심이다. 테이블이 없는 상태로 요청을 받으면 안 되므로, `await sequelize.sync()` 로 테이블을 먼저 준비한 뒤 `app.listen` 으로 서버를 연다.

- `sequelize.sync()` — 정의한 모델대로 테이블이 없으면 만든다. 6장의 `CREATE TABLE IF NOT EXISTS` 에 해당한다. 이미 있으면 건드리지 않는다.
- `Post.count() === 0` — 빈 DB일 때만 샘플을 넣는 가드다. 6장에서 `SELECT COUNT(*)` 로 같은 일을 했다. 파일 DB라 두 번째 실행부터는 이미 데이터가 있어 건너뛴다.

- `Post.bulkCreate([.])` — `create` 가 한 건이라면 `bulkCreate` 는 배열을 한 번에 넣는다. 여러 `INSERT` 를 한 번의 호출로 처리한다.
- `app.listen` 은 콜백 `main` 끝에 — 테이블 준비와 시딩이 모두 `await` 로 끝난 뒤에야 서버가 열린다. 비동기 작업의 순서를 `async/await` 로 줄 세운 결과다.

서버를 띄우고 6장과 똑같은 `curl` 을 보내 보면, 같은 엔드포인트가 같은 상태코드로 답한다. 응답 본문에 `createdAt` / `updatedAt` 이 더 보이는 것 외에는 클라이언트 입장에서 차이가 없다. 속(`SQL ↔ ORM`)을 통째로 갈았는데 겉(`API`)은 그대로 — 이게 계층을 잘 나눈 설계의 모습이다.

테이블 관계 — `hasMany` / `belongsTo` 로 잇기

지금까지는 테이블 하나(`posts`)였다. 하지만 실제 게시판은 유저가 글을 쓰고, 글에는 댓글이 달린다. 테이블이 서로 얹힌다. 이 얹힘을 다룰 때 ORM과 raw SQL의 격차가 가장 크게 벌어진다. 6장이라면 JOIN을 손으로 써야 할 일을, ORM은 `include` 한 줄로 끝낸다.

만들 구조는 `User -< Post -< Comment` 다. 한 명의 유저가 여러 글을 쓰고, 한 글에 여러 댓글이 달린다.

모델 셋과 관계 선언

```
const User = sequelize.define("User", { name: DataTypes.STRING });
const Post = sequelize.define("Post", { title: DataTypes.STRING, content: DataTypes.TEXT });
const Comment = sequelize.define("Comment", { content: DataTypes.TEXT });

// 관계 선언 — 한 명의 User가 여러 Post를, 한 Post가 여러 Comment를 가진다.
User.hasMany(Post); // users 1 : N posts (posts 테이블에 UserId 컬럼이 자동 생성됨)
Post.belongsTo(User);
Post.hasMany(Comment); // posts 1 : N comments
Comment.belongsTo(Post);
```

모델 정의가 간결해진 점부터 보자. 컬럼에 옵션이 필요 없으면 `{ name: DataTypes.STRING }` 처럼 타입만 바로 적어도 된다. `{ type: DataTypes.STRING }` 을 줄인 형태다.

핵심은 관계 선언 네 줄이다. 관계는 양쪽에서 짝으로 선언한다.

- `User.hasMany(Post)` — "한 유저가 여러 글을 가진다"(1:N). 이 선언 한 줄로 Sequelize가 `posts` 테이블에 `UserId` 외래키 컬럼을 자동으로 만든다. 6장이라면 `CREATE TABLE` 에 `user_id INTEGER REFERENCES users(id)` 를 직접 적었어야 할 부분이다.
- `Post.belongsTo(User)` — 반대쪽에서 "글은 한 유저에게 속한다"고 선언한다. `hasMany` 와 `belongsTo` 는 한 관계의 양면이라 둘 다 적어 줘야 양방향으로 데이터를 오갈 수 있다.
- `Post.hasMany(Comment)` / `Comment.belongsTo(Post)` — 글과 댓글도 같은 1:N 관계로 잇는다. `comments` 테이블에 `PostId` 외래키가 자동 생긴다.

hasMany 와 **belongsTo** , 누가 외래키를 갖나 — 외래키는 항상 **belongsTo** 쪽 테이블에 생긴다. "글은 유저에게 속한다"(`Post.belongsTo(User)`)니까 `posts` 에 `UserId` 가 붙는다. "여럿을 가진" 쪽(`User`)이 아니라 "하나에 속한" 쪽(`Post`)이 외래키를 든다 — 1:N에서 N쪽이 키를 갖는 SQL의 원칙 그대로다.

데이터 만들기 — 외래키로 잇는다

```
// 데이터 만들기 - 연결은 외래키(UserId, PostId)로 한다.
const user = await User.create({ name: "김철수" });
const post = await Post.create({ title: "첫 글", content: "내용", UserId: user.id });
await Comment.create({ content: "좋은 글이네요!", PostId: post.id });
await Comment.create({ content: "감사합니다.", PostId: post.id });
```

유저를 먼저 만든다. 그 유저의 `id` 를 글의 **UserId** 에 넣어 "이 글은 이 유저가 썼다"고 잇는다. `UserId` 는 우리가 만든 적 없는 컬럼인데, 앞서 `User.hasMany(Post)` 가 자동으로 만들어 둔 외래키다. 댓글도 같은 식으로 `PostId` 에 글의 `id` 를 넣어 글에 붙인다. 연결의 정체는 결국 외래키에 든 숫자 하나다 — 6장에서 직접 채웠을 그 값을, 여기서는 속성에 담아 넘긴다.

include — JOIN을 한 줄로

```
// include = JOIN을 대신한다. 글 하나에 작성자와 댓글을 한 번에 가져온다.
const result = await Post.findByPk(post.id, {
  include: [User, Comment],
});

console.log("글 제목:", result.title);
console.log("작성자:", result.User.name); // belongsTo → 단수 result.User
console.log("댓글 수:", result.Comments.length); // hasMany → 복수 result.Comments
console.log("댓글 내용:", result.Comments.map((c) => c.content));
```

여기가 이 장의 절정이다. 글 하나를 가져오면서 작성자와 댓글을 한 번에 끌어온다. 6장에서 같은 일을 하려면 이런 SQL을 손으로 써야 했다.

```
SELECT posts.*, users.name AS author_name, comments.content AS comment_content
FROM posts
LEFT JOIN users ON posts.user_id = users.id
LEFT JOIN comments ON comments.post_id = posts.id
WHERE posts.id = ?
```

게다가 JOIN의 결과는 평평한 행들이다. 댓글이 둘이면 같은 글 정보가 두 줄로 중복돼 나오고, 그걸 다시 코드로 "글 1개 + 댓글 배열"로 재조립하는 후처리가 필요하다. 손이 많이 간다.

ORM은 이걸 `include: [User, Comment]` 한 줄로 대신한다. 그리고 결과를 평평한 행이 아니라 중첩된 객체로 돌려준다.

- `result.User` — 단수다. `belongsTo` (글은 한 유저에 속한다)라서 작성자는 하나뿐이므로 `result.User.name` 처럼 단수로 접근한다.
- `result.Comments` — 복수다. `hasMany` (글은 여러 댓글을 가진다)라서 댓글은 배열이다. `.length` 로 세고, `.map` 으로 내용을 뽑는다.

단수/복수가 관계의 종류를 그대로 따른다 — "하나에 속하면 단수, 여럿을 가지면 복수". 이름도 모델 이름을 따라간다 (`User` , `Comments`). JOIN 결과를 직접 재조립하던 일을 ORM이 알아서 객체 트리로 만든다. 관계가 **ORM의 강점**이라고 하는 이유가 이것이다.

as 로 이름 바꾸기 — 같은 모델을 다른 의미로 두 번 이을 때(예: 글의 `author` 와 `editor` 가 둘 다 `User`), 관계에 별명을 단다. `Post.belongsTo(User, { as: "author" })` 로 선언하면 가져올 때 `include: [{ model: User, as: "author" }]` 로 적고 결과는 `result.author` 로 읽는다. 그러면 `result.User` 가 아니라 우리가 정한 `author` 라는 이름으로 작성자 객체가 실린다. 이렇게 이름을 도메인 용어에 맞추면 코드의 의미가 더 또렷해진다.

raw SQL vs ORM — 한눈에

관점	raw SQL (6장)	ORM (7장)
쓰는 언어	SQL 문자열 + 자바스크립트	자바스크립트만
테이블 생성	<code>CREATE TABLE</code> 직접 작성	모델 정의 + <code>sync()</code>
<code>id</code> · 시각 컬럼	직접 선언(<code>created_at</code> 등)	<code>id</code> / <code>createdAt</code> / <code>updatedAt</code> 자동
생성 후 결과	<code>INSERT</code> 뒤 다시 <code>SELECT</code>	<code>create</code> 가 부여된 값까지 반환
부분 수정	기존 값과 합쳐 통째로 <code>UPDATE</code>	바뀐 속성만 <code>save()</code>
관계 조회	<code>JOIN</code> 작성 + 결과 재조립	<code>include</code> 한 줄, 객체 트리 반환
DB 교체	방언별로 SQL 손봐야 함	드라이버· <code>dialect</code> 만 교체
SQL 가시성	항상 보임(우리가 씀)	숨겨짐(<code>logging</code> 으로 확인)

ORM이 무조건 우월한 건 아니다. 아주 복잡한 통계 쿼리나 DB 고유 기능을 정밀하게 쓸 때는 손으로 쓴 SQL이 더 짧고 빠르다. 그래서 실무에서도 대부분은 ORM으로 쓰되, 까다로운 한두 군데는 raw SQL로 내려가는 식으로 섞어 쓴다. 6장과 7장을 모두 익히는 이유다.

흔한 실수

- 수정 후 `save()` 를 안 부른다. → `post.title = '!'` 하지만 하고 끝내면 메모리 속 객체만 바뀌고 DB는 그대로다. `await post.save()` 까지 가야 `UPDATE` 가 나간다.
- `await` 를 빠뜨린다. → `const posts = Post.findAll()` 라고만 쓰면 `posts` 엔 데이터가 아니라 Promise가 담긴다. `findAll` 앞에 `await` 를 붙이고, 핸들러가 `async` 인지 확인한다.
- 모델 이름과 테이블 이름을 헷갈린다. → 모델은 단수(`Post`), 테이블은 복수(`posts`)다. 코드에서는 모델 이름(`Post`)으로 부른다.
- 외래키 이름을 직접 다르게 쓴다. → `hasMany / belongsTo` 가 만드는 기본 외래키는 `UserId · PostId` (모델명 + `Id`)다. 데이터를 넣을 때 이 이름과 맞춰야 연결된다.
- `board.db` 를 커밋한다. → 코드가 아니라 데이터다. `.gitignore` 의 `*.db` 로 막혀 있는지 확인한다.
- `sync()` 전에 쿼리를 날린다. → 테이블이 없어 에러가 난다. 항상 `await sequelize.sync()` 로 테이블을 준비한 뒤 데이터를 다룬다.

직접 해보기

1. 연결 설정의 `logging: false` 를 `logging: console.log` 로 바꿔 실행해 보자.
`create · findAll · save · destroy` 가 각각 어떤 SQL로 통역되는지 터미널에서 직접 확인한다.
2. 게시판 API에 작성자로 거르는 검색을 더해 보자. `GET /posts?author=김철수` 로 요청하면 그 작성자의 글만 나오게 — 힌트: `Post.findAll({ where: { author } })`. (4장의 `req.query` 와 묶어서 생각하면 쉽다.)
3. 관계 예제에서 유저 한 명의 글을 전부, 각 글의 댓글까지 한 번에 가져와 보자. 힌트: `User.findByPk(user.id, { include: { model: Post, include: Comment } })` 처럼 `include` 를 중첩한다.

정리

- ORM은 객체와 테이블 사이의 통역사다. 우리는 자바스크립트로 모델을 선언하고, Sequelize가 그걸 SQL로 옮겨 DB와 주고받는다.
- 6장에서 손으로 쓰던 SQL이 메서드로 바뀐다:
`create` (INSERT) · `findAll / findByPk` (SELECT) · `save` (UPDATE) · `destroy` (DELETE). `id` 와 `createdAt / updatedAt` 은 자동이다.
- ORM 호출은 비동기라 `await` 를 달고 다니고, 그래서 핸들러가 `async` 가 된다. 테이블은 `sync()` 로 먼저 준비한 뒤 서버를 연다.
- 관계(`hasMany / belongsTo`)를 선언하면 외래키가 자동으로 생기고, `include` 한 줄로 JOIN과 결과 재조립을 대신해 작성자·댓글을 중첩 객체로 받는다.

다음 장(08 · MVC 게시판 + 유저)에서는 지금까지 한 파일에 뭉쳐 있던 라우트·로직·DB 접근을 라우터·컨트롤러·서비스 계층으로 나눠, 규모가 커져도 흐트러지지 않는 구조를 만든다.